Alaa Ismaiel

Frontend foundations, sign-in, owner/inventory UI and integration

Branch: alaa | Plan begins when the team receives the starter.

Own the remaining frontend and coordinate the complete local demo. Keep teammates independent through shared components and contracts, and merge small working PRs throughout the two weeks.

Your code ownership

- apps/web/src/app/**, components/**, lib/**
- apps/web/src/features/auth/**
- apps/web/src/features/owner/**
- apps/web/src/features/inventory/**
- apps/web/src/styles.css; root docs/config with teammate review

Your agreed interfaces

- Sign-in uses Mustafa's /auth/login + /auth/me and Role/SessionUser contracts.
- Owner UI calls Shiam's owner endpoints; inventory UI calls Mustafa's medicine/batch endpoints.
- Use Mohammad's dashboard screen and Hamid's Preferences component through routes coordinated with Alaa; Yousef owns public screens.
- You review shared types/schema/route changes; Mustafa alone authors Prisma migrations.

Start without waiting

Pull the foundation from main into your branch. Run npm ci, copy both apps/api/.env.example and apps/web/.env.example to .env on first setup, then npm run dev. The frontend shows only a plain startup message. No database models/migrations/seed or product features are implemented. Mustafa authors the database setup; follow his instructions after that PR merges.

Read the final specification, proposed API contract, acceptance checklist and your task folders. You build the assigned pages/services yourself. There are no prebuilt guards, calculations, forms, components, adapters or fixtures. Agree interfaces early and create your own small local test data to work independently.

Scope boundary

Do not take over other members' assigned features. No deployment, billing, new stack, big design rewrite or optional challenges before CORE is integrated. Do not merge broken builds just to meet a date.

Week 1 - build your area

Days are relative, not the instructor deadline. Day 7 is a review/buffer day; adjust to your availability. Deliver small reviewed PRs throughout the week.

Day 1

Publish the foundation-only starter through a PR. Agree API shapes, schema needs, routing and ownership with the team. Create shared TypeScript wire types together; feature folders are empty and the single page is only a startup check.

Day 2

Create the application routes, navigation, responsive shell and shared styles/components yourself. Begin sign-in/logout, session handling and role-specific route protection; coordinate real JWT requests with Mustafa.

Day 3

Build the owner pharmacy list/search/status filters and create-pharmacy form from scratch. Use response types agreed with Shiam and write your own local fixtures while his API is in progress.

Day 4

Build account creation and subscription forms yourself: dates, suspend/reactivate and delete confirmation. Implement request/error handling; use local test data until Shiam's endpoints are ready.

Day 5

Build inventory search/status filters, visible count, batch detail rows and every indicator badge. Inactive subscription keeps read access but disables mutations with a clear explanation.

Day 6

Build medicine create/edit and batch create/edit forms. Read-only batch number/cost/barcode after creation; editable quantity/expiry/location. After success refresh data without page reload.

Day 7

Buffer/review: merge Week 1 PRs, check each member has visible contributions, and start real owner/inventory API integration. Record remaining blockers and avoid large late merges.

Week 2 - integrate and prove

Day 8: API integration. Day 12: core acceptance. Days 13-14: English rehearsal and fixes. No new optional features while CORE is failing.

Day 8

Connect sign-in/owner/inventory to the real API. Show 400/401/403/404/409 messages, reset stale session on unauthorized responses and use role-specific navigation.

Day 9

Complete real create/update/delete flows and validation feedback. Verify subscription changes refresh owner views and pharmacist read-only status; do not bypass backend rules with client-only checks.

Day 10

Coordinate inventory/dashboard refresh with Mohammad (navigation reload or shared invalidation). Verify low+expiring badges and counts against real API payloads, including empty inventory.

Day 11

Integrate Hamid's preference controls with minimal shared-file edits. Review Yousef's mobile/keyboard public pages; help with reusable components while preserving his ownership.

Day 12

Run docs/ACCEPTANCE.md with all members in real API integration. Confirm clean startup on another member's laptop and no .env/node_modules committed. Fix frontend defects and assign backend fixes to owners.

Day 13

Freeze feature scope, merge tested fixes and rehearse the full English demo: owner onboarding -> stock -> dashboard -> public search -> rejected action -> persisted database.

Day 14

Buffer/final review. Update README with real setup and known limitations. Prepare short presentation sections for all six people; verify demo data/time and every member's understanding.

Ready-to-merge checklist

[ ] A16: all owned screens loading/empty/error, backend validation and subscription reason; successful changes refresh views.

[ ] Private routes prompt sign-in and enforce role navigation; real integration has no fake-login or sample fallback.

[ ] Inventory supports multiple indicators and visible-item count; immutable batch fields are not submitted in PATCH.

[ ] Owner can onboard and edit/delete subscriptions while inventory remains preserved; delete confirmation is explicit.

[ ] A17/A18: another laptop can start locally; all CORE scenarios checked; every member contributes and speaks English.

Handoffs and dependencies

- Day 1: starter PR first; shared contracts/schema discussion; other members then branch from the baseline.

- Day 2-3: receive Mustafa auth/guards; frontend can use mocks until ready.

- Day 6-8: receive Shiam owner and Mustafa inventory APIs, Hamid public/profile/duty APIs and Mohammad dashboard.

- Daily: short status and small PR review; Days 8 and 12 are integration gates, not the first time anyone merges.

Every pull request

Commit only your intended files; push your branch; open a PR into main. Include the task/rule IDs, a short explanation, and tests or screenshots. Run npm run typecheck, npm test and npm run build. Backend contributors also document their endpoints in Swagger. Alaa reviews/merges; keep .env, node_modules, dist and temporary files untracked.

Your English presentation segment

Introduce the problem and product, show owner/inventory UX and loading/error feedback, then hand off to each member. Explain one UI decision and one honest remaining limitation.

Everyone should understand the whole flow: owner onboarding -> pharmacist stock -> dashboard -> public search. Be ready to explain a rule outside your own feature. Final acceptance uses real API requests and persisted PostgreSQL data; locally authored test fixtures are only for development.